

Module 3 — Python BDD RESTful Automations

Student Name	Shreya Ghosh
Course	B.Tech — Computer Science and Business Systems (CSBS)
Year	4th Year
Enrollment No.	12023052018010
Programming Language	Python
Automation Tool	Selenium WebDriver
Academic Year	2026

Experiment 1 — Implementation of Selenium Python and Behave BDD

1. Problem Statement

Develop and execute a Selenium web automation scenario using the Python Behave BDD framework. The scenario should describe a successful SauceDemo login using Gherkin language and implement the corresponding browser actions using Selenium WebDriver.

2. Objective

To set up and execute a Python BDD automation framework using Behave and Selenium WebDriver for an end-to-end web automation scenario.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Behave BDD Framework
- Google Chrome and ChromeDriver
- VS Code
- SauceDemo practice application
- Gherkin
- BDD automation
- Assertions
- `time.sleep()` for visible execution

4. Source Code / Implementation

The implementation was created using a Gherkin feature file and Python step definitions integrated with Selenium WebDriver.

Feature File — login.feature

```
Feature: SauceDemo Login

  Scenario: Successful login with valid credentials
    Given I open the SauceDemo website
    When I enter valid username and password
    And I click the login button
    Then I should see the products page
```

Python Step Definitions — login_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

@given("I open the SauceDemo website")
def open_website(context):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    context.driver.get("https://www.saucedemo.com/")
    time.sleep(2)

@when("I enter valid username and password")
def enter_credentials(context):
    context.driver.find_element(By.ID, "user-name").send_keys("standard_user")
```

```
context.driver.find_element(By.ID, "password").send_keys("secret_sauce")
time.sleep(1)
```

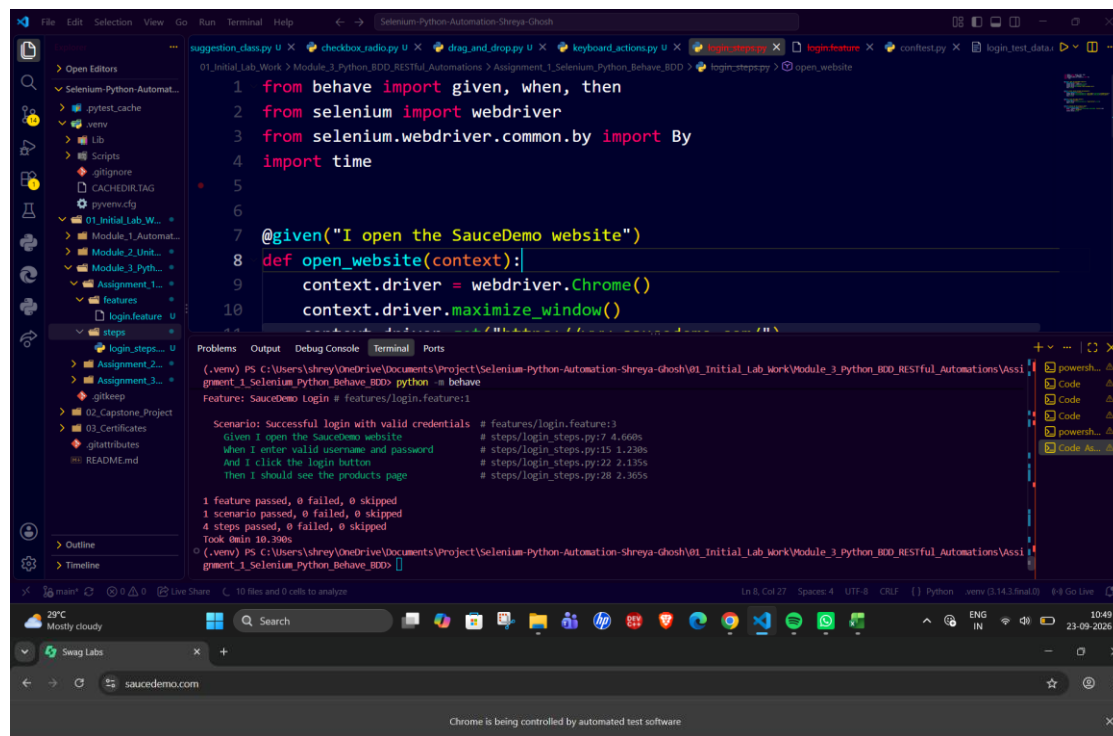
```
@when("I click the login button")
def click_login(context):
    context.driver.find_element(By.ID, "login-button").click()
    time.sleep(2)
```

```
@then("I should see the products page")
def verify_products_page(context):
    title = context.driver.find_element(By.CLASS_NAME, "title").text
    assert title == "Products"
    context.driver.quit()
```

5. Output

The SauceDemo login scenario was executed through Behave. The browser opened, valid credentials were entered, the Login button was clicked, and the Products page was verified successfully.

Execution completed successfully with 1 feature passed, 1 scenario passed, and 4 steps passed.



Swag Labs

standard_user

Login

Accepted usernames are:

standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user

Password for all users:

secret_sauce

6. Result

The Selenium Python and Behave BDD automation scenario was successfully implemented and executed. The valid login scenario was completed and the Products page was verified.

7. Observation

The Gherkin scenario describes the expected behavior using Given, When and Then steps. Behave maps these steps to Python functions, where Selenium WebDriver performs the actual browser automation.

8. Conclusion

The experiment demonstrated how Selenium WebDriver can be integrated with the Behave BDD framework to create readable, behavior-driven web automation scenarios.

Experiment 2 — Use of Test Data Driven Automation in Python Behave Framework

1. Problem Statement

Develop a test-data-driven automation using the Python Behave framework for REST API testing. The same API validation scenario should be executed with multiple user IDs supplied through a Scenario Outline and Examples table.

2. Objective

To perform Python automation using the Behave framework and demonstrate a data-driven BDD approach for REST API automation.

3. Tools, Software and Concepts Used

- Python 3
- Behave BDD Framework
- Requests library
- REST API
- VS Code
- Gherkin
- Scenario Outline
- Examples table
- JSON response validation
- Assertions

4. Source Code / Implementation

The implementation uses Behave's Scenario Outline and Examples table to execute the same REST API test with different user IDs.

Feature File — api.feature

```
Feature: REST API Data Driven Automation
```

```
Scenario Outline: Verify user details using different user IDs
```

```
    Given I send a GET request for user ID "<user_id>"
```

```
    When the API response is received
```

```
    Then the response status code should be 200
```

```
    And the response should contain user ID "<user_id>"
```

```
Examples:
```

```
| user_id |
| 1       |
| 2       |
| 3       |
```

Python Step Definitions — api_steps.py

```
from behave import given, when, then
import requests

@given('I send a GET request for user ID "{user_id}"')
def send_get_request(context, user_id):
    context.user_id = user_id
    context.response = requests.get(
        f"https://jsonplaceholder.typicode.com/users/{user_id}"
    )

@when("the API response is received")
def response_received(context):
    assert context.response is not None

@then("the response status code should be 200")
def verify_status_code(context):
    assert context.response.status_code == 200

@then('the response should contain user ID "{user_id}"')
def verify_user_id(context, user_id):
    data = context.response.json()
    assert data["id"] == int(user_id)
```

5. Output

The Scenario Outline executed the same API test for user IDs 1, 2 and 3. Each response returned status code 200 and the returned JSON data contained the expected user ID.

Execution completed successfully with 1 feature passed, 3 scenarios passed, and 12 steps passed.

```
Examples:
| user_id |
| 1       |
| 2       |

(venv) PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_3_Python_RESTful_Automations\Assignment_2_Data_Driven_Behave_Automation> python -m behave
USING RUNNER: behave.runner:Runner
ParserError: failed to parse "C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_3_Python_RESTful_Automations\Assignment_2_Data_Driven_Behave_Automation\features\api.feature":
Parser failure in state=initial at line 1: "from behave import given, when, then"
(venv) PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_3_Python_RESTful_Automations\Assignment_2_Data_Driven_Behave_Automation> python -m behave
USING RUNNER: behave.runner:Runner
Feature: REST API Data Driven Automation # features/api.feature:1

  And the response should contain user ID "1" # steps/api_steps.py:23 0.001s
  And the response should contain user ID "1" # steps/api_steps.py:23
  And the response should contain user ID "2" # steps/api_steps.py:23 0.001s
  And the response should contain user ID "2" # steps/api_steps.py:23
  And the response should contain user ID "3" # steps/api_steps.py:23 0.000s
  And the response should contain user ID "3" # steps/api_steps.py:23

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
12 steps passed, 0 failed, 0 skipped
Took 0min 1.749s
(venv) PS C:\Users\shrey\OneDrive\Documents\Project\Selenium-Python-Automation-Shreya-Ghosh\01_Initial_Lab_Work\Module_3_Python_RESTful_Automations\Assignment_2_Data_Driven_Behave_Automation>
```

6. Result

The test-data-driven REST API automation was successfully implemented using Python and Behave. All three data sets were executed and validated successfully.

7. Observation

The Examples table provides multiple test data values to a single Scenario Outline. Behave substitutes each value into the Gherkin steps and executes the same Python step definitions for every test case.

8. Conclusion

The experiment demonstrated how the Behave framework can support data-driven API automation using Scenario Outline and Examples, reducing the need to create separate scenarios for each data value.

Experiment 3 — Selenium Page Object Model in the Python Behave Framework

1. Problem Statement

Develop a Selenium web automation scenario using the Behave BDD framework and Page Object Model (POM). The automation should use Gherkin language to describe the test scenario and a Page Object class to separate page locators and UI operations from the test steps. The same login scenario should be executed with multiple sets of test data using a Scenario Outline.

2. Objective

To implement and execute a Selenium-based web automation using the Behave BDD framework, Gherkin language, and Page Object Model, while using test data to execute the login scenario with different valid users.

3. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Behave BDD Framework
- Google Chrome and ChromeDriver
- VS Code
- SauceDemo practice application
- Page Object Model (POM)
- Gherkin
- Scenario Outline
- Examples table for test data
- Assertions
- `time.sleep()` for visible execution

4. Source Code / Implementation

The implementation integrates Selenium WebDriver with Behave and uses a LoginPage class to contain page locators and UI methods.

Feature File — login.feature

```
Feature: SauceDemo Login using Page Object Model
```

```
Scenario Outline: Login with different valid users
```

```
Given I open the SauceDemo login page
```

When I login using username "<username>" and password "<password>"
Then I should see the Products page

Examples:

username	password	
standard_user	secret_sauce	
problem_user	secret_sauce	

Python Step Definitions — login_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

class LoginPage:

    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")
    PRODUCTS_TITLE = (By.CLASS_NAME, "title")

    def __init__(self, driver):
        self.driver = driver

    def open(self):
        self.driver.get("https://www.saucedemo.com/")

    def login(self, username, password):
        self.driver.find_element(*self.USERNAME).send_keys(username)
        self.driver.find_element(*self.PASSWORD).send_keys(password)
        self.driver.find_element(*self.LOGIN_BUTTON).click()

    def get_page_title(self):
        return self.driver.find_element(*self.PRODUCTS_TITLE).text

@given("I open the SauceDemo login page")
def open_login_page(context):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    context.login_page = LoginPage(context.driver)
    context.login_page.open()
    time.sleep(2)

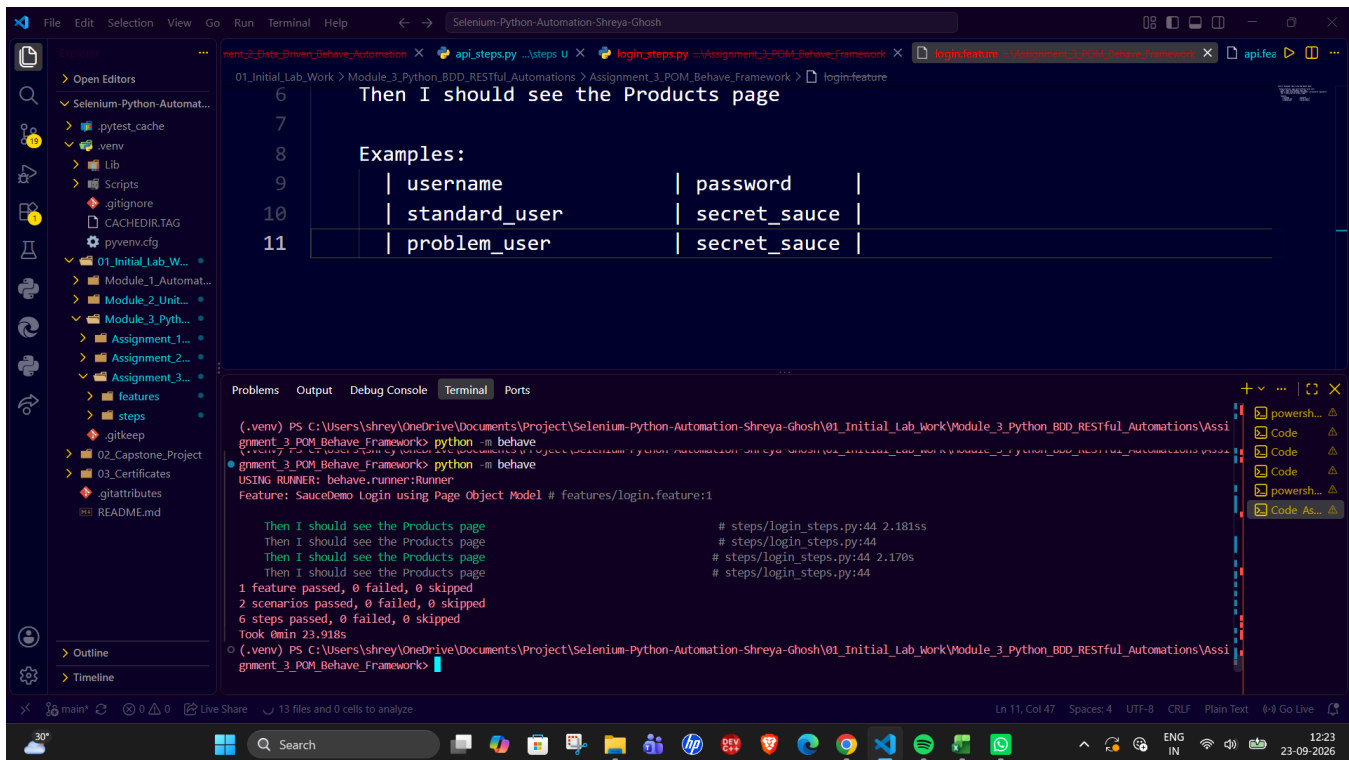
@when('I login using username "{username}" and password "{password}"')
def login_with_credentials(context, username, password):
    context.login_page.login(username, password)
    time.sleep(2)

@then("I should see the Products page")
def verify_products_page(context):
    assert context.login_page.get_page_title() == "Products"
    context.driver.quit()
```

5. Output

The Behave feature was executed using the Scenario Outline with two sets of valid login credentials. For each test-data row, Selenium opened the SauceDemo login page, entered the supplied credentials, clicked the Login button, and verified that the Products page was displayed.

Execution completed successfully with 1 feature passed, 2 scenarios passed, and 6 steps passed.



6. Result

The Selenium Page Object Model automation was successfully implemented using the Python Behave framework. Both data-driven login scenarios were executed successfully and the Products page was verified.

7. Observation

The LoginPage class contains the page locators and UI methods, while the Behave step definitions describe the actions performed during the test. The Scenario Outline and Examples table allow the same login workflow to be executed with different test data.

8. Conclusion

The experiment demonstrated the integration of Selenium WebDriver, Page Object Model, Gherkin and Behave. Separating page interactions into the LoginPage class improves the organization of the automation, while the Scenario Outline supports repeated execution using different test data.